Doc. no: P1349R0
Audience: LEWG, SG1
Date: 2018-11-06
Reply-To: Vinnie Falco (vinnie.falco@gmail.com)

Better Integration of Sender Executors

Contents

1. Overview

This document proposes changes to P1194r0 [1] which allow it to better integrate with the proposed Executors design described in P0443r9 [2].

2. Motivation and Scope

P1194r0 describes a new executor interface allowing the implementation of Future-like asynchrony without the overhead of synchronization typically required when `std::promise::set_value` may be called concurrently with `std::future::get`. We fully support this goal, and assume that the paper delivers on the claim that the proposed executor interfaces `make_value_task` and `make_bulk_value_task` achieve it.

However, the paper also proposes harmful changes to P0443r9 which affect performance, and proposes an integration into P0443r9 which conflicts with the design of Executors. In this paper we highlight the problems with P1194r0 stemming from a misunderstanding of Executors, roll back the harmful changes, and propose a clean way to add lazy executors on top of P0443r9.

3. Impact on the Standard

This is a pure library proposal. It does not add any new language features, nor does it alter any existing standard library headers. However, this library also requires the library features offered in P0443r9.

4. Discussion

In this section we review the problematic passages from P1194r0 by quoting each passage and describing the issues. We assume that readers are already familiar with P0443r9 and P1194r0.

- *"It achieves this by replacing P0443r9's six `Executor::*execute`..."*

  The *Executor* concept does not prescribe any execution functions, it only specifies the base requirements of *CopyConstructible*, *Destructible*, and *EqualityComparable*. The execution functions referred to by P1194 are actually part of the interface of each interface concept. P0443r9 defines two interface concepts: *OneWayExecutor* and *BulkOneWayExecutor*. We note that these interface concepts are orthogonal. A particular executor can support zero, one, or both concepts. An executor is not required to implement all interfaces.

- *"Note: This paper only seeks to add support for lazy task composition and execution to P0443 while maintaining feature parity with P0443."*

  The paper (P1194r0) goes beyond adding support for lazy task composition and attempts to simplify the existing interface concepts by expressing them in terms of a "grand unified theory" of execution which uses lazy execution as the basis operations.

- *"The `then_execute` API on the Executor concept is renamed `make_value_task`, and `bulk_then_execute` is renamed `bulk_make_value_task`."*

  As stated earlier, the *Executor* concept does not specify any execution functions. The `then_execute` and `bulk_then_execute` interfaces are actually part of the *ThenExecutor* and *BulkThenExecutor* concepts which were removed prior to P0443r9 and placed into P1244r0 [3].

- *"The following Executor APIs are removed: `execute`, `twoway_execute`, `bulk_execute`, and `bulk_twoway_execute`."*

  As stated earlier, the *Executor* concept does not specify any execution functions. The interfaces `exceute` and `bulk_execute` are part of the *OneWayExecutor* and *BulkOneWayExecutor* concepts in P0443r9. The `twoway_execute` and `bulk_twoway_execute` are part of the *TwoWayExecutor* and *BulkTwoWayExecutor* concepts which were removed prior to P0443r9 and placed into P1244r0.

- *"...the Executor concept gets a `submit` API..."*

  As stated earlier, the *Executor* concept only contains requirements which are common to all interface concepts. Forcing all executors, current and future, to implement `submit` is an overly broad and unnecessary requirement to implementing lazy execution.

- *"We do not expect any performance difference between the two forms of the one way execute definition..."*

This statement is partly false. A performance difference appears when a lazy executor is type-erased using the polymorphic wrapper, and used to simulate the behavior of the *OneWayExecutor*. It is the author's view that the use of a type-erased lazy executor to perform a one way execution cannot avoid incurring an additional memory allocation.

While the paper continues on to refer incorrectly to interfaces of the *Executor* concept which do not exist, we refer to the statements above for why these references are incorrect.

A stated goal of P1194r0 is to simplify the fundamental concepts involved in asynchronous execution. In addition to performance considerations, we believe the proposed change of basis operations actually makes user-defined executors more difficult to write. Feedback from SG1 during Rapperswil anticipated a "zoo of [user-defined] executors," a position with which we agree. The *OneWayExecutor* concept in P0443r9 is much simpler to implement than a sender executor concept, as can be seen by comparing two hypothetical implementations:

```
//
// Models OneWayExecutor
//
struct inline_executor
{
  friend bool operator==(const inline_executor&, const inline_executor&) noexcept
  {
    return true;
  }

  friend bool operator!=(const inline_executor&, const inline_executor&) noexcept
  {
    return false;
  }

  template <class Function>
  void execute(Function f) const noexcept
  {
    f();
  }
};

//
// Models SenderExecutor
//
// Note: This implementation should be taken with a grain
//       of salt, as the specification in P1196 is insufficient
//       to produce a complete, working implementation.
//
struct inline_executor {

  template <class F, class R>
  struct __inline_receiver {
    F f_;
    R r_;
    template <class... Args>
    void set_value(Args&&... args) {
      std::move(r_).set_value(
        std::move(f_)(std::forward<Args>(args)...)
      );
    }
    template <class E>
    void set_error(E&& e) {
      std::move(r_).set_error(std::forward<E>(e));
    }
    void set_done() {
      std::move(r_).set_done();
    }

    static constexpr void query(std::experimental::execution::receiver_t) { }
  };

  template <class T, class E=std::exception_ptr>
  struct __subject {
    // Ed: omitted for brevity
    // ...
  };

  template <class S, class F>
  struct __task_submit_fn {
    S s_;
    F f_;

    template <typename... Values>
    struct _value_types_helper {
      using type = std::invoke_result_t<F, Values...>;
```

```
    };

    static constexpr void query(std::experimental::execution::sender_t) noexcept { }

    template <class Receiver>
    void submit(Receiver&& r) {
      std::move(s_).submit(
        __inline_receiver<F, std::decay_t<Receiver>>{std::move(f_), std::forward<Receiver>(r)}
      );
    }

    auto executor() const { return inline_executor{}; }
  };

  template <class NullaryFunction>
  void execute(NullaryFunction&& f) const {
    f();
  }

  template <std::experimental::execution::ReceiverOf<inline_executor> R>
  void submit(R&& r) const {
    std::forward<R>(r).set_value(*this);
  }

  template <std::experimental::execution::Sender S, class Function>
  auto make_value_task(S&& s, Function f) const {
    return __task_submit_fn<std::decay_t<S>, Function>{
      std::forward<S>(s), std::move(f)
    };
  }

  using sender_desc_t = std::experimental::execution::sender_desc<std::exception_ptr, inline_executor>;
  static constexpr sender_desc_t query(std::experimental::execution::sender_description_t) { return { }; }
  static constexpr void query(std::experimental::execution::sender_t) { }
};
```

While we are in generally in favor of using a single, more universal primitive to express multiple execution strategies we do not believe that the resulting complexity pushed onto users justifies adopting the sender executor model as that universal primitive.

As shown above, P1194r0 currently has structural problems which prevent it from being seriously considered. But can we fix the problems by rigorously adopting P0443r9's interface properties in a way that preserves the lazy execution features? The answser is of course yes, and the next section explains how.

5. Proposed Changes

1. Introduce two new concepts, *SenderExecutor* and *BulkSenderExecutor*

   These two concepts are in addition to the *OneWayExecutor* and *BulkOneWayExecutor* concepts already described in P0443r9. Adding concepts as refinements of *Executor* is the prescribed method of adding additional executor models. This can be seen in P1124r0 which adds *TwoWayExecutor*, *BulkTwoWayExecutor*, *ThenExecutor*, and *BulkThenExecutor*. After Executors ships, new executor models may continue to be added as refinements. We believe the extensible system of interface properties which allows for both compile-time and runtime introspection of executor capabilities is a remarkably elegant and flexible design which caters to the strengths of C++.

2. Add the `submit` and `make_value_task` interfaces as requirements of *SenderExecutor*.

   As the `execute` interface is part of the *OneWayExecutor* refinement, so should the `submit` and `make_value_task` interfaces be part of the *SenderExecutor* refinement. The behavior of these interfaces remains the same as described in the paper.

3. Add a new interface property type `sender_t`.

   As the `oneway_t` interface property type described in P0443r9 is used to require, prefer, or query an executor for the one-way execution interface capability, so should the `sender_t` interface property be defined to determine the sender execution interface capability. A possible implementation for the property may look like this:

```
    // SenderExecutor interface property
    struct sender_t
    {
        static constexpr bool is_requirable = true;
        static constexpr bool is_preferable = false;

        template <class... SupportableProperties>
        class polymorphic_executor_type;

        using polymorphic_query_result_type = bool;

        template <class Executor>
        static constexpr bool static_query_v = implementation-defined
```

```
        static constexpr bool value() const { return true; }
    };
    static constexpr sender_t sender;
```

4. Add the `submit` and `make_bulk_value_task` interfaces as requirements of *BulkSenderExecutor*.

   As the `bulk_execute` interface is part of the *BulkOneWayExecutor* refinement, so should the `submit` and `make_bulk_value_task` interfaces be part of the *BulkSenderExecutor* refinement. The behavior of these interfaces remains the same as described in the paper.

5. Add a new interface property type `bulk_sender_t`.

   As the `bulk_oneway_t` interface property type described in P0443r9 is used to require, prefer, or query an executor for the bulk one-way execution interface capability, so should the `bulk_sender_t` interface property be defined to determine the bulk sender execution interface capability. A possible implementation for the property may look like this:

```
    // BulkSenderExecutor interface property
    struct bulk_sender_t
    {
        static constexpr bool is_requirable = true;
        static constexpr bool is_preferable = false;

        template <class... SupportableProperties>
        class polymorphic_executor_type;

        using polymorphic_query_result_type = bool;

        template <class Executor>
        static constexpr bool static_query_v = implementation-defined

        static constexpr bool value() const { return true; }
    };
    static constexpr bulk_sender_t bulk_sender;
```

The implementation of a particular lazy executor should add hooks for the `require`, `prefer`, and `query` customization points described in P0443r9. Users who desire a lazy executor should use the aforementioned customization points to obtain an executor with the lazy execution feature. This example shows how a generic algorithm which depends on lazy execution might acquire lazy executors:

```
template <typename Executor>
void perform (const Executor& ex)
{
    // change ex to a SenderExecutor
    auto const lazy_ex = require(ex, sender);

    // change ex to a BulkSenderExecutor
    auto const bulk_lazy_ex = require(ex, bulk_sender);

    …
```

If the changes described in this section are not adopted, then at the very least we would like to see the following in a future revision of P1194r0:

- Correct references to requirements of *Executor* which are actually requirements of individual refinements such as *OneWayExecutor* and *BulkOneWayExecutor*.
- Demonstrate with code that a polymorphic wrapper used to type-erase a sender executor does not incur an unnecessary memory allocation when used to perform a one-way execute.
- Provide the rationale for why these changes are not adopted.

6. Acknowledgements

We thank Christopher Kohlhoff for reviewing the proposed changes for accuracy, for editorial improvements, and for example implementations.

7. References

[1] http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1194r0.html

[2] http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0443r9.html

[3] http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1244r0.html